

Problème du logarithme discret

Chiffre de GAMAL

8 octobre 2026 – B. COLOMBEL

Le compte-rendu de ce TP sera fait sous la forme d'un notebook Jupyter. Il sera à déposer sous le nom

Nom_Prenom-TP5.ipynb

dans le dossier relatif à ce TP dans AMETICE.

Sommaire

1	Problème du logarithme discret	1
1.1	Fonction indicatrice d'Euler (rappels et compléments)	1
1.2	Racine primitive — logarithme discret	2
1.3	Logarithme discret	3
2	Chiffre d'El Gamal	3
2.1	Génération des clés	4
2.2	Chiffrement	4
2.3	Déchiffrement	4

1 Problème du logarithme discret

1.1 Fonction indicatrice d'Euler (rappels et compléments)

Définition 1

$a \in \mathbb{N}$ est inversible modulo n si, et seulement si, il existe $b \in \mathbb{Z}_{/n\mathbb{Z}}$ tel que $a \times b \equiv 1 \pmod{n}$.

Théorème 1

1. Les seuls éléments inversibles de $\mathbb{Z}_{/n\mathbb{Z}}$ sont les éléments premiers avec n .
2. Si p est premier, tout élément non nul de $\mathbb{Z}_{/p\mathbb{Z}}$ est inversible.

Définition 2

On note $\varphi(n)$ le nombre d'éléments inversibles de $\mathbb{Z}_{/n\mathbb{Z}}$ (Indicateur d'Euler).

Remarque. Si p est premier alors, $\varphi(p) = p - 1$.

Théorème 2

Si a et n sont premiers entre eux, alors $a^{\varphi(n)} \equiv 1 \pmod{n}$.

En conséquence, on obtient l'inverse de a modulo n :

$$a^{-1} \equiv a^{\varphi(n)-1} \pmod{n}$$

Ce calcul peut paraître séduisant mais, malheureusement, cela suppose que $\varphi(n)$ a été préalablement calculé, c'est à dire que n a été factorisé. Or, la factorisation de n est de complexité exponentielle. Ce résultat a donc peu d'intérêt pratique sauf si l'on dispose d'une autre méthode plus pratique pour calculer $\varphi(n)$.

1.2 Racine primitive — logarithme discret

On considère un nombre premier p .

Dans $\mathbb{Z}/p\mathbb{Z}$, il est facile de définir une fonction exponentielle :

$$x \mapsto a^x \pmod{p}$$

On connaît même un algorithme efficace pour calculer $a^x \pmod{n}$ (algorithme des carrés) dont voici une version récursive :

```
def puissance(a, x, n):  
    """methode récursive, rapide de calcul de a^x mod n  
    Attention au cas x == 0"""  
    if x == 0:  
        return 1  
    elif x == 1:  
        return a  
    elif x % 2 == 0:  
        return puissance(a**2 % n, x // 2, n) % n  
    elif x != 1:  
        return a * puissance(a**2 % n, (x-1) // 2, n) % n
```

Notre problématique est de savoir si on peut définir sa fonction réciproque dans $\mathbb{Z}/p\mathbb{Z}$, ce qui revient à vérifier que $x \mapsto a^x$ est bien bijective de $\mathbb{Z}/p\mathbb{Z}$ dans $\mathbb{Z}/p\mathbb{Z}$.

Pour cela, on utilise la fonction `cycle(a, p)` qui retourne la liste des puissances de a^n modulo p pour n allant de 0 à $p-1$:

```
def cycle(a, p):  
    """retourne a**0, a**1, a**2, ... a**(p-1) modulo p """  
    resultat = []  
    for d in range(p-1):  
        resultat.append(puissance(a, d, p))  
    return resultat
```

Dans la suite, on prendra $p = 31$.

Exercice 1.

1. Afficher les résultats de la fonction `cycle(a, p)` avec $a = 31$ et $a = 2, a = 3, a = 4$ et $a = 5$.
2. Que se passe-t-il pour $a = 2$? La fonction $x \mapsto 2^x$ est-elle bijective? Pour quelles autres valeurs de a , observe-t-on le même comportement?
3. Quelle est la seule valeur de a parmi celles testées pour laquelle la fonction $x \mapsto a^x \pmod{31}$ est bijective?

En réalité, il y a plusieurs valeurs de a qui rendent la fonction $x \mapsto a^x \pmod{31}$ bijective : 3, 11, 12, 13, 17, 21, 22 et 24.

Dans ce cas, on parle de **racine primitive**. Plus formellement :

Définition 3

Soit $n \in \mathbb{N}$ et $\varphi(n)$ la fonction indicatrice d'Euler. Un entier a avec $1 < a < n$ est une *racine primitive* de n si :

- a est premier avec n
- pour tout d avec $1 \leq d < \varphi(n)$, $a^d \not\equiv 1 \pmod{n}$.

Exercice 2. Écrire une fonction `primitive(a, p)` qui teste si a est une racine primitive de p avec p premier.

Exemples d'utilisation.

```
In[10]: primitive(3, 31)  
Out[10]: True
```

```
In[11]: primitive(4, 31)
Out[11]: False
```

Exercice 3.

1. Écrire une fonction `racine_primitive(p)` qui recherche une racine primitive de p avec p premier.

Exemples d'utilisation.

```
In[14]: racine_primitive(4999)
Out[14]: 3
```

```
In[15]: racine_primitive(1000021)
Out[15]: 11
```

2. Écrire une fonction `liste_racines_primitives(p)` qui retourne la liste des racines primitives de p avec p premier.

```
In[19]: liste_racines_primitives(31)
Out[19]: [3, 11, 12, 13, 17, 21, 22, 24]
```

1.3 Logarithme discret

Théorème 3

Soit p un nombre entier premier et a une racine primitive de p .

La fonction définie sur $\mathbb{Z}/p\mathbb{Z}$ vers $\mathbb{Z}/p\mathbb{Z}$ par :

$$a \mapsto a^x \pmod{p}$$

est bijective et admet une inverse.

Définition 4

Soit p un nombre entier premier et a une racine primitive de p .

La fonction inverse de la fonction

$$f : \begin{array}{ccc} (\mathbb{Z}/p\mathbb{Z})^* & \rightarrow & (\mathbb{Z}/p\mathbb{Z})^* \\ a & \mapsto & a^x \end{array}$$

est le *logarithme népérien en base a modulo p* .

On le note $\log_a \pmod{p}$.

Exemple 1. On sait que 3 est une racine primitive de 31. On a $\log_3(19) = 4 \pmod{31}$.

En effet,

$$3^4 \equiv 81 \equiv 2 \times 31 + 19 \equiv 19 \pmod{31}$$

2 Chiffre d'El Gamal

Le système cryptographique d'El Gamal est un système asymétrique (à clé publique comme RSA) mis au point par Taher El Gamal en 1984.

Il repose fondamentalement sur la difficulté de résoudre le problème du logarithme discret.

Rappel : dans $\mathbb{Z}/p\mathbb{Z}$ avec p premier, il est facile de définir la fonction exponentielle $x \mapsto a^x \pmod{p}$. Son calcul est rapide en utilisant la méthode d'exponentiation rapide (algorithme des carrés). Par contre, pour de grandes valeurs de p , le calcul du logarithme discret est un problème « difficile ».

Bob veut envoyer un message chiffré à Alice. Alice doit d'abord générer ses clés (publique et privée).

2.1 Génération des clés

La première étape du schéma de chiffrement consiste à produire une paire de clés : la clé publique et la clé secrète (ou clé privée). La première servira à chiffrer les messages et la deuxième à les déchiffrer.

Pour générer sa paire de clés, Alice choisit d'abord un nombre premier p et une racine primitive de p que l'on note g (comme générateur).

Alice choisit alors *au hasard* un nombre $a \in \mathbb{Z}/p\mathbb{Z}$ et calcule $g^a = A$.

Sa clé publique est (p, g, A) et sa clé privée est a .

2.2 Chiffrement

Pour chiffrer un message m encodé comme un élément de $\mathbb{Z}/p\mathbb{Z}$, Bob choisit au hasard un entier b compris entre 0 et $p - 2$.

Le message est chiffré en $c = mA^b$ (c comme chiffré)

Pour permettre à Alice de retrouver le message clair, Bob calcule $B = g^b$ et envoie le couple (c, B) .

2.3 Déchiffrement

Le calcul suivant permet à Alice, à l'aide de sa clé privée a , de retrouver le message :

$$c \times (B^a)^{-1} \pmod{p}$$

En effet,

$$\begin{aligned} c \times (B^a)^{-1} &\equiv mA^b \times ((g^b)^a)^{-1} \pmod{p} \\ &\equiv m(g^a)^b \times (g^{ba})^{-1} \pmod{p} \\ &\equiv m \times g^{ab} \times (g^{ba})^{-1} \pmod{p} \\ &\equiv m \pmod{p} \end{aligned}$$

Exercice 4. Écrire une fonction `clefs(k)` qui permet de générer une paire de clés El Gamal, avec un module compris entre 2^k et $2^{k+1} - 1$. Votre fonction devra retourner le couple `(public_key, private_key)`.

Pour écrire votre fonction, vous pourrez procéder comme suit :

- choisir au hasard un nombre premier p compris entre 2^k et $2^{k+1} - 1$;
- choisir une racine primitive de p ;
- choisir un nombre au hasard b compris entre 0 et $p - 2$;
- calculer B .

Vous serez peut-être amené à utiliser la fonction `randprime` de la bibliothèque `sympy` et la fonction `randint` de la bibliothèque `random`.

Exemple d'utilisation.

```
In[25]: k = 16
        public_key, private_key = clefs(k)
        print(public_key, private_key)
Out[25]: (81343, 6, 44949) 43833
```

Exercice 5. Écrire une fonction `chiffrer(m, public_key)` qui permet de chiffrer un nombre m (compris entre 0 et $p - 1$) avec le chiffrement d'El Gamal et la clé `public_key`.

Exemple d'utilisation.

```
In[27]:(p, g, A) = public_key
        m = randint(0, p-1)

        c, B = chiffrer(m, public_key)
        print(c, B)
Out[27]: 3548029743 5885
```

Exercice 6. Écrire une fonction `dechiffrer(c, B, private_key, private_key)` qui permet de déchiffrer le message chiffré (c, B) avec le chiffrement d'El Gamal.

Exemple d'utilisation.

```
In[29]: res = dechiffrer(c, B, private_key)
        res == m
Out[29]: True
```

Fondamentalement, le chiffrement d'El Gamal s'applique à des nombres. Mais très souvent, les messages qu'on souhaite chiffrer sont plutôt des textes! Pour chiffrer des textes, il faut donc passer par une étape préalable de codage des caractères qui le composent.

Il y a plusieurs manières de Procéder. Voici celle que nous proposons ici :

1. Chaque caractère est encodé par son code ASCII décimal, composé de 1, 2 ou 3 chiffres. Étant donné un caractère `c`, le code ASCII correspondant est obtenu par `ord(c)`.
2. Les codes ASCII des caractères sont transformés en `str`, puis ramenés sur 3 chiffres en ajoutant si nécessaire des 0 à gauche.
3. Les chaînes représentant les codes ASCII sur 3 chiffres sont concaténées les unes à la suite des autres pour obtenir une seule (grande) chaîne, ensuite transformée en `int`.

Exercice 7.

1. Écrire une fonction `encode(texte)` en un nombre entier en suivant le procédé indiqué au-dessus.
2. Écrire une fonction `decode(nombre)` effectuant l'opération réciproque : étant donné un nombre entier nombre, retrouver la chaîne de caractères texte qu'il représente.
Pour cela, on peut utiliser la fonction `chr`. Étant un code ASCII donné `num`, le caractère associé est obtenu par `chr(num)`.

Astuce. Pour récupérer les codes ASCII des différents caractères, vous pouvez peut-être utiliser le reste (%) et le quotient (//) de la division par 1000.

3. Tester les fonctions. Pour tout message `m`, on doit avoir

`decode(encode(m)) = m`

Exercice 8. À ce stade, vous pouvez chiffrer/déchiffrer des messages entre vous.

- Un voisin vous communique sa clé publique. Vous pouvez donc maintenant lui envoyer des messages chiffrés.
- Choisir un message texte ((très) court! puisque le nombre encodant votre message ne doit pas dépasser son module). Le convertir en nombre. Chiffrer ce nombre. Envoyer le cryptogramme résultant à votre voisin.
- Votre voisin déchiffre le cryptogramme. Puis il décode le nombre obtenu, et retrouve votre texte.

Exercice 9. (Bonus)

Pour pouvoir échanger des messages aussi longs que vous voulez, écrire une fonction qui découpe les entiers trop longs pour en faire plusieurs messages.